Lab Report – Whisper STT Implementation

Author: Elif Yurdakul · **Model:** whisper-1 · **Audio:** DARE oral history interview (1:41 min, mono, 44.1 kHz, 1.1 MB)

1. How the system works

`whisper_transcription.py` checks its prerequisites (API key from `.env`, ffmpeg), loads the audio with pydub and runs four stages. **Basic transcription:** the first 60 s are sent with `response_format="verbose_json"`, which returns segment-level start/end times, and `language="en"` to avoid language misdetection. **Guided vs. unguided:** the same clip is transcribed with and without a prompt and compared with `difflib` (word similarity, word-level differences, punctuation). **Chunking:** the audio is split into fixed-length chunks (mono, 16 kHz, 64 kbps), each with its start offset recorded. Each chunk is transcribed, the offset is added to every segment so timestamps refer to the full recording, and the end of the previous chunk's text is passed as context to the next one. **Export:** TXT (`[HH:MM:SS] text`), SRT subtitles and JSON. Rate-limit and timeout errors are retried with exponential backoff, and a failing chunk is logged without stopping the run. Total cost of all experiments: about $0.035.

2. Prompted vs. unprompted transcription

Prompt	Word similarity	Punctuation	Differences
Context: description of the interview's topic and setting	100 %	17 → 17	none
Style: "Umm, let me think, like, hmm... Okay, so, uh..."	98.3 %	17 → 22	kept "um"; "in personnel" → "the impersonel" ❌

- **The context prompt changed nothing.** The speech is clear and contains no unusual names or terms, so Whisper had nothing to correct. Context prompts help with ambiguity (names, jargon, poor audio), not with clean everyday speech.
- **The style prompt changed how the text was written.** Whisper continues in the prompt's style: it kept the filler word "um", which it normally removes, and added more pauses as commas. The result is closer to verbatim speech, which is valuable for linguistic or oral history work.
- **There is a trade-off.** The verbatim style also introduced a non-word ("impersonel"). The clean default was the more accurate version. The prompt should therefore match the purpose: clean style for meeting notes and search, verbatim style for research.

3. Benefits of chunking

Chunking removes the 25 MB / single-request limit, so a 2-hour meeting becomes a series of small, independent requests. One failed chunk can be retried without repeating the whole file, and long jobs show progress. The 1:41 recording was split into four 30-second chunks (95–235 KB each). After offset correction, each chunk's first segment starts at exactly 00:00:30, 00:01:00 and 00:01:30, confirming the timestamps refer to the original recording.

4. Challenges

- **Environment:** pydub depends on `audioop`, which Python 3.13+ removed; it was fixed with `audioop-lts`. ffmpeg had to be installed with Homebrew. Several open VS Code windows and an active Conda `base` environment caused commands to run in the wrong place.
- **Blind chunk cuts:** cuts every 30 s happened to fall into pauses here, but a cut inside a word would damage both chunks.
- **Long segments:** some segments last 17–24 s, too long for readable subtitles.
- **No speaker labels:** fast interviewer–speaker turns ("Thirty-nine?" / "No, no, no...") appear without attribution.

5. Recommendations for improving accuracy

1. **Prompts by purpose:** use glossaries of participant names, company terms and acronyms for business meetings; choose clean or verbatim style deliberately.
2. **Smarter chunking:** split at silences (`pydub.silence.detect_silence`) or overlap chunks by a few seconds and de-duplicate.
3. **Subtitle quality:** request word-level timestamps and build subtitle lines of at most about 7 s.
4. **Speaker diarization:** add a diarization step, or a model that supports it, so meeting transcripts show who said what.
5. **Human review:** spot-check names and numbers, because even a well-prompted transcript can contain plausible-looking errors.